PHD PROGRAM

Applied Data Science & Artificial Intelligence

UNIVERSITÀ DEGLI STUDI DI TRIESTE

Explainable Artificial Intelligence

Rule-based explainable artificial intelligence



Introduction

Two main paradigms of rule-based XAI:

1. **Rule-based models**

   Machine learning models that learn **decision rules** to make predictions based on input features, e.g. decision trees. In their basic form, these models are simple and intrinsically explainable

2. **Rule-based post-hoc XAI methods**

   Post-hoc, explainable artificial intelligence (XAI) methods that add explainability to already trained machine learning models by extracting **decision rules**

Decision Rules

IF condition THEN prediction

The basic decision rule is a simple **IF-THEN** statement, which consists of a condition (or antecedent) and a prediction (consequent), for example:

IF it rains today and IF it is April (condition), THEN it will rain tomorrow (prediction)

Multiple rules can be combined (with **AND/OR** operators) to make predictions

In machine learning the decision rules are learned through an algorithm

Decision Rules

The usefulness of a decision rule can be expressed by two quantities: **support** and **accuracy**

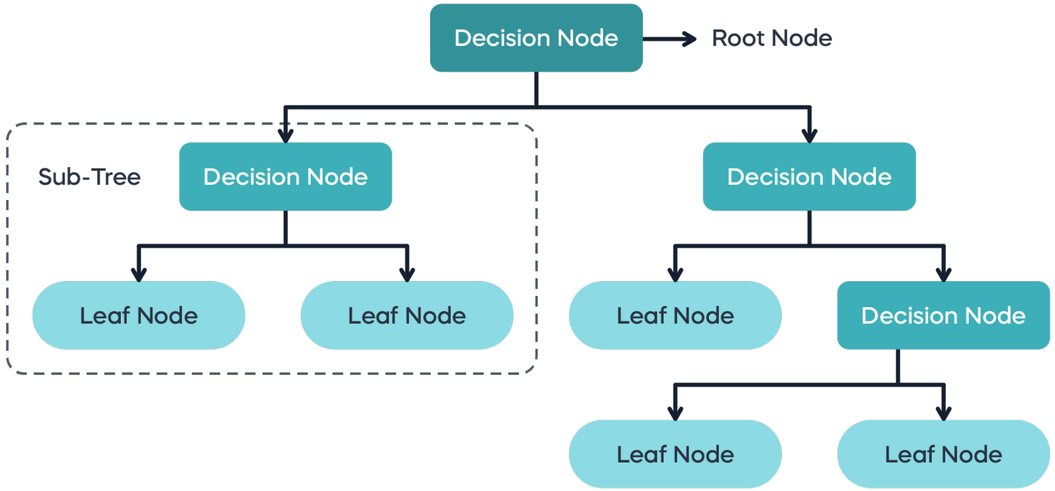
- **Support (or coverage)**: the percentage of instances to which the condition applies
- **Accuracy (or confidence)**: measures how accurate the rule is in predicting the correct class for the instances to which the condition applies

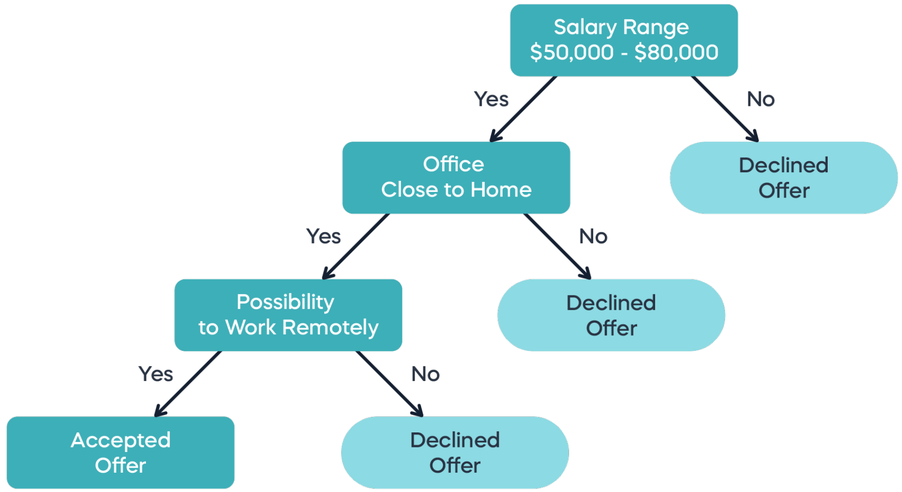
Usually there is a trade-off between accuracy and support: adding more features to the condition, accuracy increases but support decreases

Rule-based models: Decision Trees (Quick recap)

Decision trees are a popular and expressive technique to learn decision rules from data

Nodes of the tree are either **decision nodes** or **leaf nodes**; the former represent **splits** of the target variable, based on a specific predictor and condition, while the latter are used to make **predictions**





source https://365datascience.com/tutorials/machine-learning-tutorials/decision-trees/

Rule-based models: Decision Trees (Quick recap)

Building a decision tree: how can we decide which variable to branch out at each level?

- Binary splitting, performed top-down and greedy
- Variable selection criteria (where the complexity lies), mainly two approaches:
  - Entropy
  - Gini Index

Entropy is a measure of disorder or impurity of a node and is computed as:

$$E = -\sum_{i=1}^{n} p_i \, log_2 \left(p_i\right) \qquad E \in [0,1]$$

Where n is the number of classes and p_i is the probability of randomly selecting a point in class i

Leaf nodes have an entropy of 0 if all instances belongs to one class and of 1 if the classes are divided equally

Rule-based models: Decision Trees (Quick recap)

Gini Index measures the probability to misclassify a random instance. The lower the Gini Index, the lower the likelihood of misclassification. Considering k classes in the target variable, the Gini Index is computed as:

$$Gini = 1 - \sum_{i=1}^{k} P(i)^2 \qquad Gini \in [0,0.5]$$

Where $P(i)$ represents the ratio of observations in class i over the total number of observations in the node. A Gini Index of 0 corresponds to the highest level of purity, whereas a Gini Index of 0.5 indicates a random assignment of classes.

Information Gain is computed by subtracting the average value at child nodes from the one of the parent node and is used to select splits. The goal is to maximize the information gain that results from each split point

Rule-based post-hoc XAI methods

We will discuss the following rule-based post-hoc XAI methods:

	MODEL AGNOSTIC	TREE MODELS
GLOBAL	○ **Trepan** (Craven and Shavlik, 1995) https://github.com/AImenes/TREPAN	○ **CMM** (Domingos, 1998) ○ **inTrees** (Deng, 2014) ○ **STA** (Zhou et al., 2016)
LOCAL	○ **LORE** (Guidotti et al., 2018) ○ **ANCHORS** (Ribeiro et al., 2018)	○ **BELLATREX** (Dedja et al., 2023)

Trepan

1995

Extracting Tree-Structured Representations of Trained Networks

Mark W. Craven and Jude W. Shavlik
Computer Sciences Department
University of Wisconsin-Madison
1210 West Dayton St.
Madison, WI 53706
craven@cs.wisc.edu, shavlik@cs.wisc.edu

Abstract

A significant limitation of neural networks is that the representations they learn are usually incomprehensible to humans. We present a novel algorithm, TREPAN, for extracting comprehensible, symbolic representations from trained neural networks. Our algorithm uses queries to induce a decision tree that approximates the concept represented by a given network. Our experiments demonstrate that TREPAN is able to produce decision trees that maintain a high level of fidelity to their respective networks while being comprehensible and accurate. Unlike previous work in this area, our algorithm is general in its applicability and scales well to large networks and problems with high-dimensional input spaces.

MAIN IDEA

Use the trained network as a “labelling **oracle**” and derive a concept description that is comprehensible yet classifies instances in the same way as the network

Trepan

The algorithm can be applied to trained **neural network models**. It works as a “knowledge extractor”: the derived decision-trees describe the global model learned by the neural network, not why an individual instance was labelled as such.

Some key characteristics of the Trepan algorithm are:

The oracle

- while building a tree, Trepan queries the oracle to get the class of instances (as predicted by the network)

Tree expansion

- expand node with greatest potential to increase the fidelity of the extracted tree to the network, i.e. the node *n* with largest fraction of misclassified nodes *f(n) = reach(n)(1 - fidelity(n))*

Split types

- the role of internal nodes in a decision tree is to partition the input space to increase the separation of instances of different classes; splits based on m-of-n expressions (at least m over n conditions satisfied)

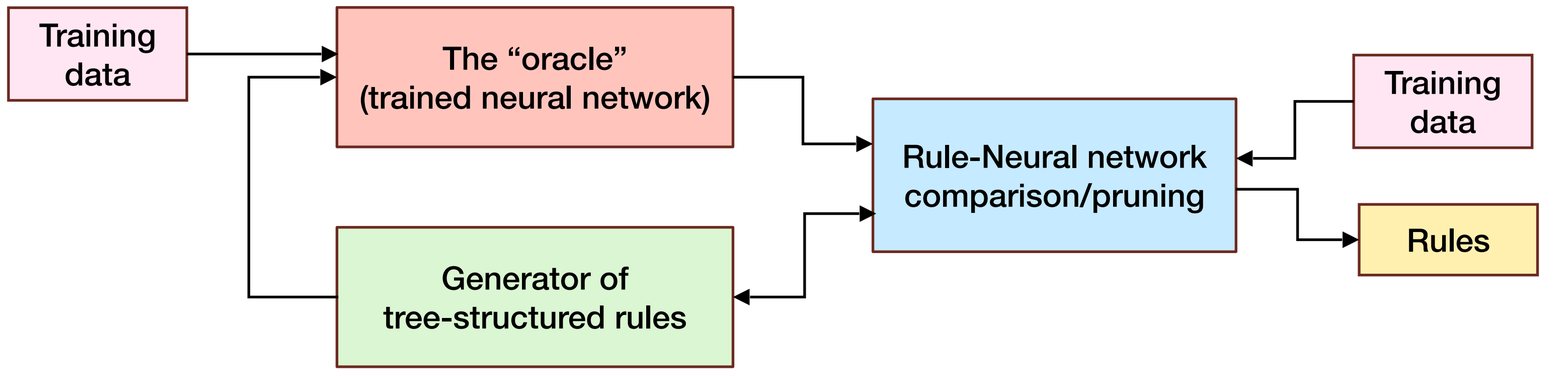
Trepan

Split selection

- thanks to the oracle, Trepan uses as many instances as desired, overcoming the limitation of conventional tree-induction algorithms where the amount of training data used to select splits decreases with the depth of the tree

Stopping criteria

- a node becomes a leaf if single class with high probability or reaches the maximum number of internal nodes



LORE

2018

Local Rule-Based Explanations of Black Box Decision Systems

Riccardo Guidotti
ISTI-CNR & University of Pisa, Italy
riccardo.guidotti@isti.cnr.it

Anna Monreale
University of Pisa, Italy
anna.monreale@unipi.it

Salvatore Ruggieri
University of Pisa, Italy
salvatore.ruggieri@unipi.it

Dino Pedreschi
University of Pisa, Italy
dino.pedreschi@unipi.it

Franco Turini
University of Pisa, Italy
franco.turini@unipi.it

Fosca Giannotti
ISTI-CNR of Pisa, Italy
fosca.giannotti@isti.cnr.it

ABSTRACT

The recent years have witnessed the rise of accurate but obscure decision systems which hide the logic of their internal decision processes to the users. The lack of explanations for the decisions of black box systems is a key ethical issue, and a limitation to the adoption of machine learning components in socially sensitive and safety-critical contexts. In this paper we focus on the problem of black box outcome explanation, i.e., explaining the reasons of the decision taken on a specific instance. We propose **LORE**, an agnostic method able to provide interpretable and faithful explanations. **LORE** first leans a local interpretable predictor on a synthetic neighborhood generated by a genetic algorithm. Then it derives from the logic of the local interpretable predictor a meaningful explanation consisting of: a decision rule, which explains the reasons of the decision; and a set of counterfactual rules, suggesting the changes in the instance's features that lead to a different outcome. Wide experiments show that **LORE** outperforms existing methods and baselines both in the quality of explanations and in the accuracy in mimicking the black box.

MAIN IDEA

First learn a local interpretable predictor on a synthetic neighborhood generated by a genetic algorithm, then derive a meaningful explanation from the logic of the local interpretable predictor

LORE

LORE (LOcal Rule-based Explanations), is a **model agnostic method** to extract local explanations

The methods generates a local explanation consisting of two parts:

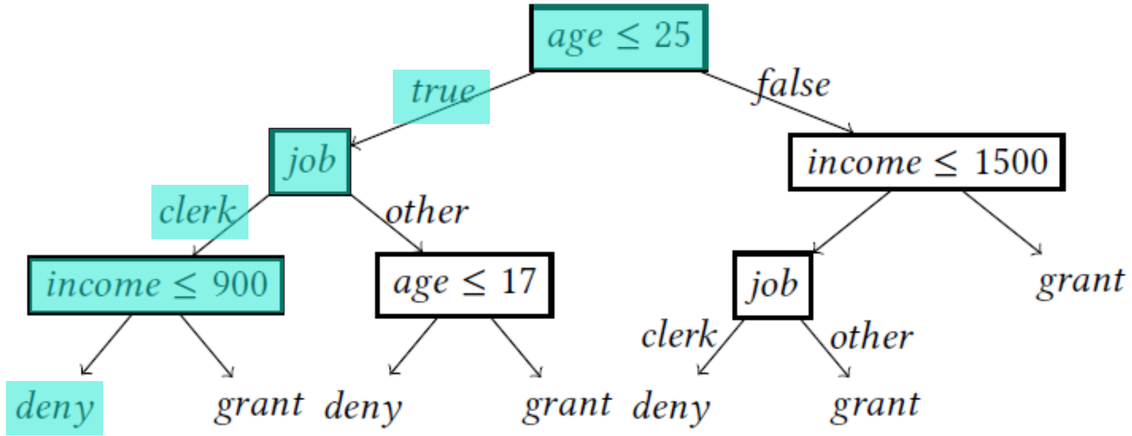
- a **decision rule**, which explains the reasons of the decision
- a **set of counterfactual rules (topic of the next lecture)**, suggesting which changes in the instance's features would lead to a different outcome

The algorithm consists in the following main steps:

1. given a black box model b and instance x, build an interpretable predictor c for b by first generating a set Z of N neighbour instances of x through a genetic algorithm (fitness tuned to generate points z close to x but different from x, with $b(x) = b(z)$ or $b(x) \neq b(z)$ in two runs of the GA).
2. from the neighbourhood Z of x, build an interpretable predictor c trained on the instances $z \in Z$ labelled with the black box model $b(z)$; the predictor should mimic the behaviour of b locally in the Z neighbourhood
3. derive from the structure of c (which is a tree) the local explanation, consisting of a single rule r and a set of counterfactual rules Φ

LORE

As an example, consider the decision tree in the figure and the following instance $x = \{(\text{age}, 22), (\text{job}, \text{clerk}), (\text{income}, 800)\}$ for which the decision **deny** (e.g., of a loan) must be explained



The decision rule extracted from the path is $\{\text{age} \leq 25, \text{job} = \text{clerk}, \text{income} \leq 900\} \rightarrow \text{deny}$

As an example, two counterfactual instances of x are:

$\{(\text{age}, 22), (\text{job}, \text{clerk}), (\text{income}, 900 + \epsilon)\}$, and $\{(\text{age}, 22), (\text{job}, \text{other}), (\text{income}, 800)\}$

ANCHORS

2018

Anchors: High-Precision Model-Agnostic Explanations

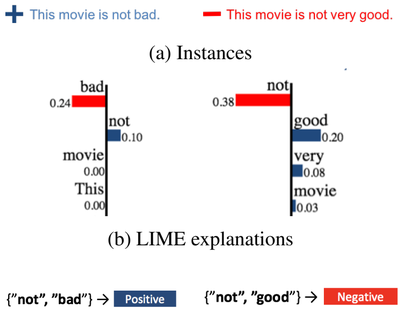
Marco Tulio Ribeiro
University of Washington
marcotcr@cs.washington.edu

Sameer Singh
University of California, Irvine
sameer@uci.edu

Carlos Guestrin
University of Washington
guestrin@cs.washington.edu

Abstract

We introduce a novel model-agnostic system that explains the behavior of complex models with high-precision rules called *anchors*, representing local, "sufficient" conditions for predictions. We propose an algorithm to efficiently compute these explanations for any black-box model with high-probability guarantees. We demonstrate the flexibility of anchors by explaining a myriad of different models for different domains and tasks. In a user study, we show that anchors enable users to predict how a model would behave on unseen instances with less effort and higher precision, as compared to existing linear explanations or no explanations.



MAIN IDEA

Learn rules with high coverage and precision growing B rules and optimising their precision using a (multi-arm) bandit algorithm, then picking the one with largest coverage.

Anchors

Anchors is another interpretation method for classification models created by Marco Tulio Ribeiro, the inventor of LIME.

It's a technique used in XAI for creating interpretable **explanations** of individual predictions made by machine learning models.

ANCHORS generate high-precision, human-understandable **rules** (anchors) that explain the behavior of complex models, particularly focusing on **local interpretability**.

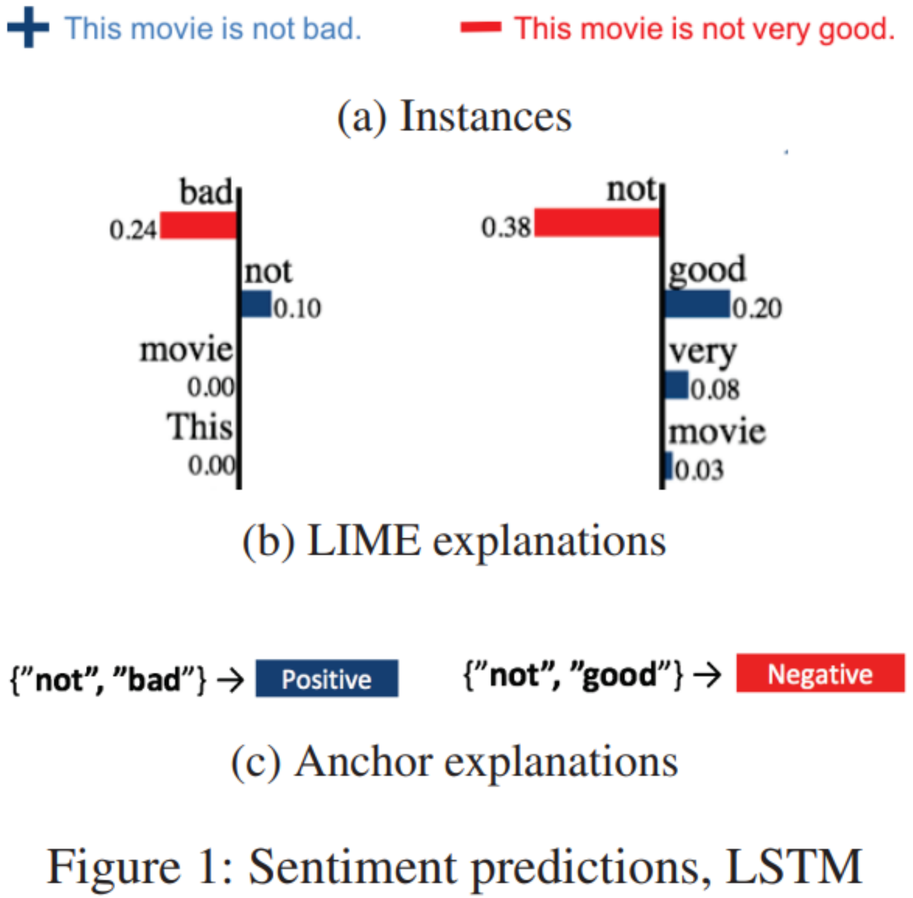


Figure 1: Sentiment predictions, LSTM

Benefits of ANCHORS

- **Interpretability:** Provides clear, human-readable explanations.

- **Local Explanations:** Focuses on explaining individual predictions rather than the entire model, making it useful for understanding specific decisions.

- **High Precision:** Ensures that the explanations are accurate and reliable.

Anchors - Important concepts

Precision:

- Precision refers to the accuracy of the anchor in predicting the same output as the model. An anchor with high precision means that when the conditions of the anchor are met, the model's prediction is almost always the same.

Coverage:

- Coverage indicates the proportion of instances for which the anchor's conditions apply within the dataset. Higher coverage means the anchor conditions are met for a larger subset of the data.

Anchor Conditions:

- These are the specific criteria or conditions that define the anchor. For example, in a classification task, an anchor might be a rule like "if age > 30 and income > $50,000, then predict 'approved'."

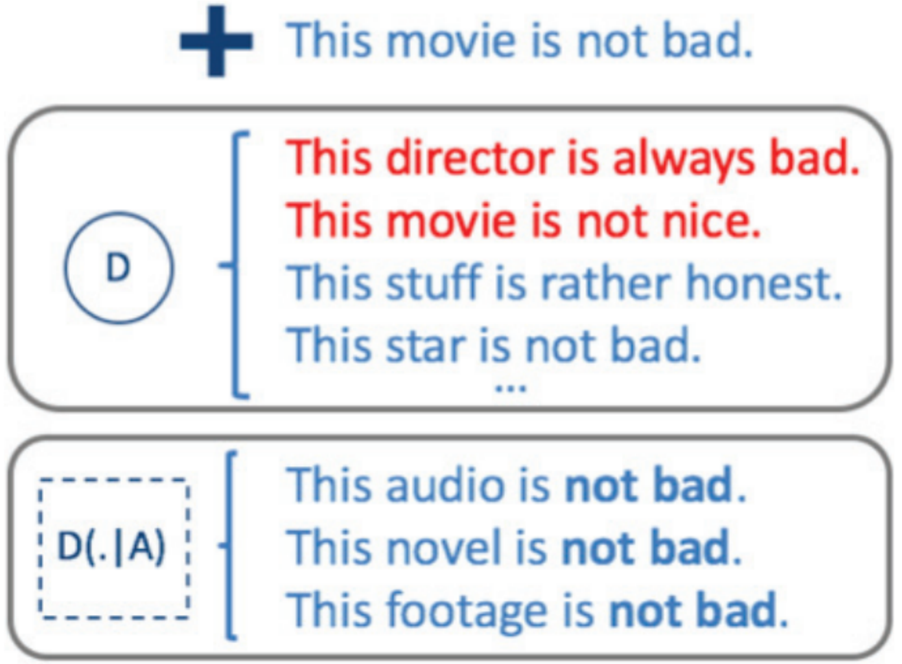
Anchors as High-Precision Explanations

Let ***D*** be the "pertubation distribution" of an instance ***x.***

Let ***A*** be a rule (set of predicates) acting on such an interpretable representation, such that ***A(x)*** returns 1 if all its feature predicates are true for instance ***x***. For example, **x = "This movie is not bad."**, ***f(x)*** = Positive, ***A(x)***=1 where ***A*** = {"not", "bad"}.

Let ***D(·|A)*** denote the conditional distribution when the rule ***A*** applies (e.g. similar texts where "not" and "bad" are present).

A is an anchor if ***A(x)***=1 and ***A*** is a sufficient condition for ***f(x)*** with high probability — in the example, if a sample ***z*** from ***D(z|A)*** is likely predicted as Positive (i.e. ***f(x)*** = ***f(z)***).



(a) $\mathcal{D}$ and $\mathcal{D}(.|A)$

Anchors as High-Precision Explanations

Formally ***A*** is an anchor if,

$$\mathbb{E}_{\mathcal{D}(z|A)}\left[\mathbb{1}_{f(x)=f(z)}\right] \geq \tau, A(x) = 1$$

, where **f(x)** is the individual prediction of a black box model ***f*** for instance ***x***.

Anchors as High-Precision Explanations

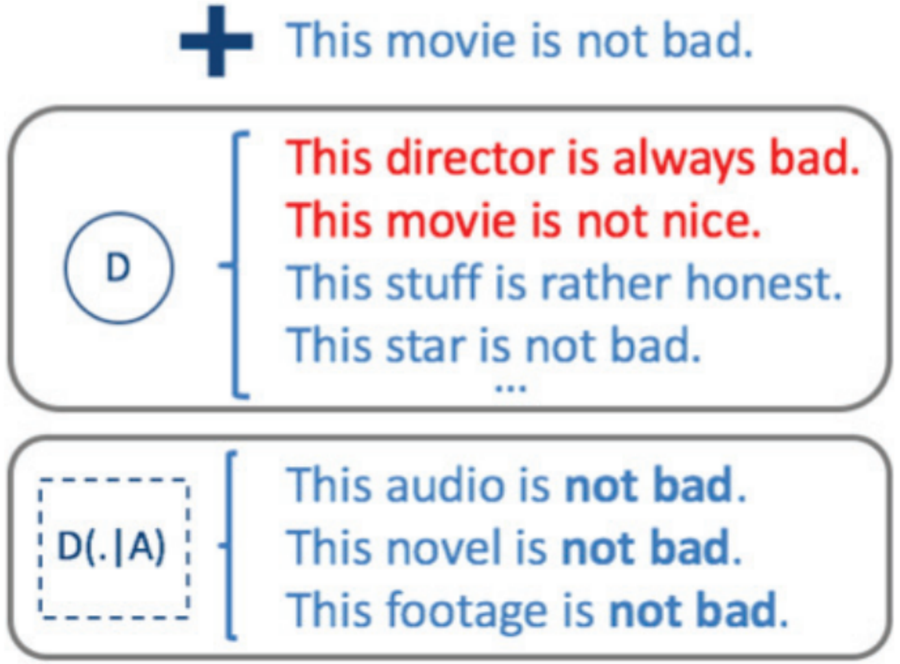
In this case, the features used by the model are uninterpretable. The interpretable representation used is the presence of individual tokens (words) in the instance.

The perturbation distribution D replaces “absent” tokens by random words with the same POS tag with probability proportional to their similarity in an embedding space (i.e. the generated sentences are coherent, and of the same length).

The anchor **A = {“not”, “bad”}** is easy to apply:

- **if the words “not” and “bad” are in the sentence, the model will predict “positive”.**

- **if either word (or both) are not present, we know we do not have sufficient information to know what the model will do.**



(a) $\mathcal{D}$ and $\mathcal{D}(.|A)$

Anchors - Structured prediction

When the output of the algorithm is a structure, the anchor approach can be used to explain any function of the output.

In Table 1, Anchors explain the predictions of a part-of-speech tagger for the word “play” in different contexts, where the tags for neighbouring words are included as part of the interpretable representation.

The anchors demonstrate that the model is picking up on reasonable patterns of the English language.

- **if play is preceded by a determiner, it is likely used as a noun.**

Instance	If	Predict
I want to play(V) ball.	previous word is PARTICLE	play is VERB.
I went to a play(N) yesterday.	previous word is DETERMINER	play is NOUN.
I play(V) ball on Mondays.	previous word is PRONOUN	play is VERB.

Table 1: Anchors for Part-of-Speech tag for the word “play”

Anchors: High-Precision Model-Agnostic Explanations

Anchors - Structured prediction

Classification of data in tabular form (categorical and/or continuous features) is a popular application of ML.

The anchors provide valuable insight into these models. Marital status appears in many different anchors for predicting whether a person makes > $50K annually (adult dataset).

The anchors for predicting recidivism for individuals released from prison (rcdv dataset), show that this model is unfair if used for bail decisions, as race and gender feature prominently.

For predicting whether a loan on the Lending Club website will turn out bad, i.e. late payment or default (lending dataset), the FICO score is sufficient in the extremes, but loan amount is taken into consideration otherwise.

	If	Predict
adult	No capital gain or loss, never married	$\leq$ 50K
	Country is US, married, work hours $>$ 45	$>$ 50K
rcdv	No priors, no prison violations and crime not against property	Not rearrested
	Male, black, 1 to 5 priors, not married, and crime not against property	Re-arrested
lending	FICO score $\leq$ 649	Bad Loan
	649 $\leq$ FICO score $\leq$ 699 and \$5,400 $\leq$ loan amount $\leq$ \$10,000	Good Loan

Table 3: Generated anchors for Tabular datasets

Anchors: High-Precision Model-Agnostic Explanations

Anchors - Image Classification

An inspection of the (admittedly bizarre) images from D(z|A) in ***Figure c*** for which the model predicts “beagle” with high confidence illustrates that attributes that humans would consider essential for a beagle prediction (legs, not being underwater, not being in the sky, not having a human body) are not quite as essential to the neural network.

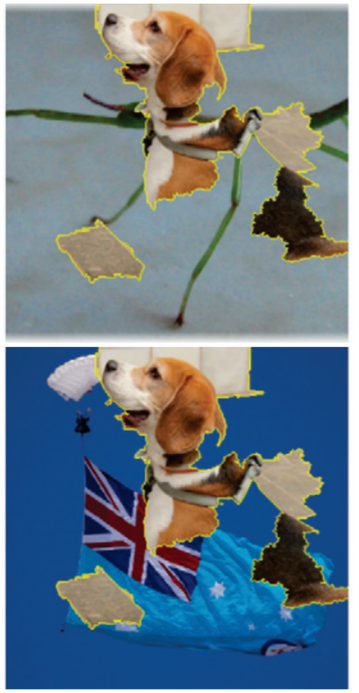


(a) Original image



(b) Anchor for “beagle”





(c) Images where Inception predicts $P(\text{beagle}) > 90\%$

Anchors: High-Precision Model-Agnostic Explanations

Efficiently Computing Anchors

Given a blackbox classifier ***f***, instance ***x***, distribution ***D***, and the desired level of precision ***τ*** , an anchor ***A*** is a set of feature predicates on ***x*** that achieves ***prec(A) ≥ τ*** , where:

$$\text{prec}(A) = \mathbb{E}_{\mathcal{D}(z|A)}\left[\mathbb{1}_{f(x)=f(z)}\right].$$

For an arbitrary ***D*** and black-box model ***f***, it is intractable to compute this precision directly. Instead, we introduce a probabilistic definition: **anchors satisfy the precision constraint with high probability**.

$$P\left(\text{prec}(A) \geq \tau\right) \geq 1 - \delta$$

If multiple anchors meet this criterion, those that describe the behavior of a larger part of the input space are preferred, i.e. ones with the largest **coverage**. Formally, the coverage of an anchor can be defined as the probability that it applies to samples from ***D***, i.e:

$$\text{cov}(A) = \mathbb{E}_{\mathcal{D}(z)}[A(z)]$$

Efficiently Computing Anchors

We thus define this search for an anchor as the following combinatorial optimization problem:

$$\max_{A \text{ s.t. } P(\text{prec}(A) \geq \tau) \geq 1-\delta} \text{cov}(A)$$

Bottom-up Construction of Anchors

Incrementally construct the anchor ***A***, which is initialised with an empty rule, i.e. one that applies to every instance. In each iteration, it generates a number of candidate rules that extend ***A*** by one additional feature predicate, $\{a_i\}$ in its definition, i.e. the set of candidate rules in each iteration is:

$$\mathcal{A} = \{A \wedge a_i, A \wedge a_{i+1}, A \wedge a_{i+2}, \ldots\}$$

It identifies the candidate rule with the highest estimated **precision**, replace ***A*** with the selected candidate, and repeat. If the current candidate rule meets the anchor definition, we have identified our desired anchor and terminate.

In order to select the best candidate rule in each iteration, we want to estimate the precision of these candidates efficiently. Since it's not possible to compute the true **precision**, we rely on samples from ***D(·|A)*** to estimate the precision of ***A***; however, a fixed number of samples may be too many or too few for an accurate estimation. Instead, we are interested in a minimal set of calls to ***f*** (fewest samples from ***D***) in order to estimate which candidate rule has the highest true **precision**.

Bottom-up Construction of Anchors

This problem can be formulated as an instance of **pure exploration multi-armed bandit problem**, i.e. each candidate ***A*** is an arm, the true precision of ***A*** on ***D(·|A)*** is the latent reward, and each pull of the arm ***A*** is an evaluation of $\mathbb{1}_{f(x)=f(z')}$ on a sample from ***D(z|A)***.

For such a setting, the **KL-LUCB** algorithm can be used to identify the rule with the highest precision. The algorithm works by constructing confidence regions based on **KL divergence**.

In each step, the algorithm selects two distinct rules: the best mean ***(A)*** and the highest upper bound ***(A')***, and updates their bounds by getting a sample each from ***D(z|A)*** and ***D(z'|A')***, and computing $\mathbb{1}_{f(x)=f(z)}$ and $\mathbb{1}_{f(x)=f(z')}$. This sampling process continues until the lower bound on ***A*** is higher than ***A'***s upper bound with tolerance $\epsilon \in [0, 1]$ If $A*$ is the arm with highest true precision, the following holds for the true precision of the chosen rule ***A***:

$$P(\text{prec}(A) \geq \text{prec}(A^*) - \epsilon) \geq 1 - \delta$$

Bottom-up Construction of Anchors

Algorithm 1 Identifying the *Best* Candidate for Greedy

function GenerateCands($\mathcal{A}, c$)
  $\mathcal{A}_r = \emptyset$
  for all $A \in \mathcal{A}; a_i \in x, a_i \notin A$ **do**
    if $\text{cov}(A \wedge a_i) > c$ **then** {Only high-coverage}
      $\mathcal{A}_r \leftarrow \mathcal{A}_r \cup (A \wedge a_i)$ {Add as potential anchor}
  return $\mathcal{A}_r$ {Candidate anchors for next round}

function BestCand($\mathcal{A}, \mathcal{D}, \epsilon, \delta$)
  initialize $\text{prec}, \text{prec}_{ub}, \text{prec}_{lb}$ estimates $\forall A \in \mathcal{A}$
  $A \leftarrow \arg\max_A \text{prec}(A)$
  $A' \leftarrow \arg\max_{A' \neq A} \text{prec}_{ub}(A', \delta)$ {δ implicit below}
  while $\text{prec}_{ub}(A') - \text{prec}_{lb}(A) > \epsilon$ **do**
    sample $z \sim \mathcal{D}(z|A), z' \sim \mathcal{D}(z'|A')$ {Sample more}
    update $\text{prec}, \text{prec}_{ub}, \text{prec}_{lb}$ for A and A'
    $A \leftarrow \arg\max_A \text{prec}(A)$
    $A' \leftarrow \arg\max_{A' \neq A} \text{prec}_{ub}(A')$
  return A

Beam-Search for Anchor Construction

Although the greedy approach we have described so far can find short anchors with the guarantee that, for each step, the choice was near optimal with high probability, it has two major shortcomings:

- Due to the greedy nature of the approach, it is only able to maintain a single rule at a time (that it incrementally augments), and thus any suboptimal choice is irreversible.
- The greedy algorithm is not directly concerned with the coverage of the anchors, and instead returns the shortest anchor that it finds.

The Beam-search approach is similar in structure to the greedy approach, with a set of ***B*** current candidates instead of a single one. After generating all the possible candidates rules, we select the ***B***-best candidates to keep based on the **KL-LUCB** approach with multiple arms (the Explore-m setting).

Beam-Search for Anchor Construction

function BeamSearch($f, x, \mathcal{D}, \tau$)
  hyperparameters B, ϵ, δ
  $A^* \leftarrow$ **null**, $\mathcal{A}_0 \leftarrow \emptyset$ {Set of candidate rules}
  loop
    $\mathcal{A}_t \leftarrow$ GenerateCands($\mathcal{A}_{t-1}$, cov(A^*))
    $\mathcal{A}_t \leftarrow$ B-BestCand($\mathcal{A}_t, \mathcal{D}, B, \delta, \epsilon$) {LUCB}
    if $\mathcal{A}_t = \emptyset$ **then break loop**
    for all $A \in \mathcal{A}_t$ s.t. $\text{prec}_{lb}(A, \delta) > \tau$ **do**
      if cov(A) > cov(A^*) **then** $A^* \leftarrow A$
  return A^*

In short, ANCHORS:

1. **Generating Candidate Conditions**:
   - The algorithm starts by generating candidate conditions that might explain the model's prediction. These conditions are based on the features of the input data.
2. **Evaluating Precision and Coverage**:

   •The algorithm evaluates each candidate condition for precision and coverage. The goal is to find conditions that consistently lead to the same prediction (high precision) and apply to a substantial portion of the data (reasonable coverage).
3. **Selecting the Best Anchor**:
   - The best anchor is selected based on a trade-off between precision and coverage. The final anchor should have high precision while covering enough instances to be meaningful.

CMM

1998

Knowledge Discovery Via Multiple Models

Pedro Domingos
Sec. Sistemas, Dept. Eng. Mecânica
Instituto Superior Técnico
Av. Rovisco Pais
Lisbon 1096, Portugal
E-mail: pedrod@gia.ist.utl.pt

Abstract

If it is to qualify as knowledge, a learner's output should be accurate, stable and comprehensible. Learning multiple models can improve significantly on the accuracy and stability of single models, but at the cost of losing their comprehensibility (when they possess it, as do, for example, simple decision trees and rule sets). This article proposes and evaluates CMM, a meta-learner that seeks to retain most of the accuracy gains of multiple model approaches, while still producing a single comprehensible model. CMM is based on reapplying the base learner to recover the frontiers implicit in the multiple model ensemble. This is done by giving the base learner a new training set, composed of a large number of examples generated and classified according to the ensemble, plus the original examples. CMM is evaluated using C4.5RULES as the base learner, and bagging as the multiple-model methodology. On 26 benchmark datasets, CMM retains on average 60% of the accuracy gains obtained by bagging relative to a single run of C4.5RULES, while producing a rule set whose complexity is typically a small multiple (2–6) of C4.5RULES's, and also improving stability. Further studies show that accuracy and complexity can be traded off by varying the number of artificial examples generated.

MAIN IDEA

Use a trained ensemble of simple models to label new generated examples, then learn the base model with a training set composed of this generated data and the original samples

→ maintain the accuracy of multiple model approaches, yet producing a single comprehensible model

CMM

CMM (Combined Multiple Models) can be applied to a trained **ensemble of simple and comprehensible models** (base learners) that perform classification

In the paper, the base learner is the C4.5 algorithm [2], which generates decision trees

The CMM method can be summarized as follows:

1) Learn m models $M_1, \ldots, M_m$ on different variations of the training set
2) Randomly generate n examples and label each of them accordingly to the ensemble of $M_1, \ldots, M_m$, conditional on individual rules and proportionally to the coverage x confidence of each rule, i.e. for any branch of each tree M_j, generate a number of examples $p\frac{n}{m}$ proportional to the fraction p of examples in the leaf, by fixing some features according to the rule and randomly sampling the others.
3) Learn a model M on the union of the training set and the generated examples

The resulting model should achieve accuracy and stability like the ensemble, while still producing explainable output

inTrees

2014

Interpreting Tree Ensembles with inTrees

Houtao Deng
California, USA

Abstract

Tree ensembles such as random forests and boosted trees are accurate but difficult to understand, debug and deploy. In this work, we provide the inTrees (interpretable trees) framework that extracts, measures, prunes and selects rules from a tree ensemble, and calculates frequent variable interactions. An rule-based learner, referred to as the simplified tree ensemble learner (STEL), can also be formed and used for future prediction. The inTrees framework can applied to both classification and regression problems, and is applicable to many types of tree ensembles, e.g., random forests, regularized random forests, and boosted trees. We implemented the inTrees algorithms in the "inTrees" R package.

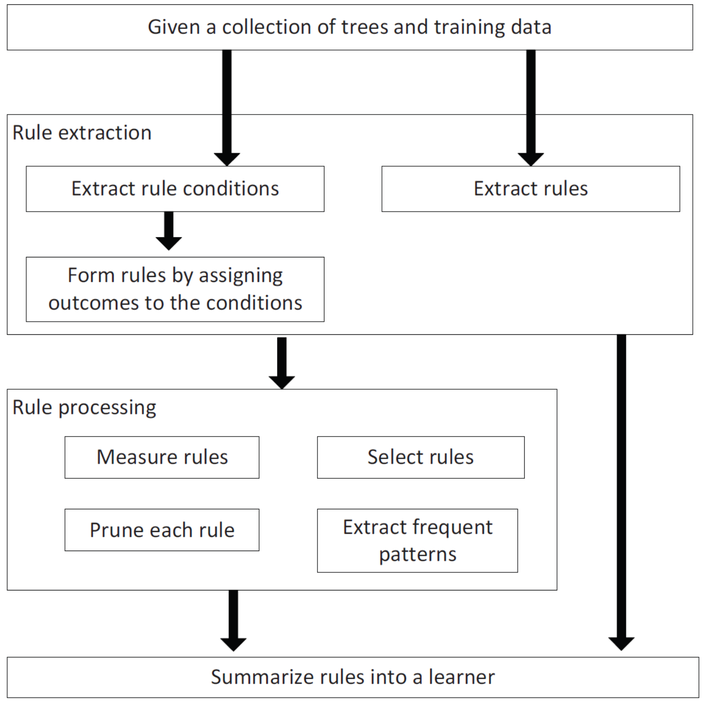
MAIN IDEA

Extract rules from a learned tree ensamble and rank them according to frequency and length. The goal is to find the most representative rules that the ensemble has acquired (large coverage in the ensemble), yet are short and interpretable.

https://sites.google.com/site/houtaodeng/intrees

inTrees

The aim of inTrees is to extract, measure, prune and select rules from a **tree ensemble** (random forests, xgboost, gbm), for either classification or regression problems



Two main stages:

1) **Rule extraction**: extract conditions with associated output by querying the data
2) **Rule processing**: measure, prune, select and find frequent patterns in the rules

Rule extraction

A set of rules is extracted from the decision tree, by **turning each branch of the tree into a rule**. Essentially, we follow the steps below:

i) the algorithm receives in input a **rule set** (initially null), a **node** (initially the root of a tree) and a **conjunction of variable-value pairs** (initially null)

ii) if the current node is a leaf, a prediction is associated with the node, the rule set is updated with the current rule and returned

iii) otherwise, for every child of the current node:

- the conjunction of variable-value pairs is updated and used as the Boolean check to split the current node
- the rule set is defined by the return of an iterative call to the rule extractor passing in the rule set, the node child and the variable-value pairs conjunction

iv) once the iterative calls are completed, the rule set is returned

Rule processing

Rules are **ranked** by quality using the following metrics:

i) **frequency**: proportion of data instances satisfying the rule condition

ii) **error** (for classification problems): ratio of incorrect over correct predictions

iii) **error** (for regression problems): mean square error between the target values of the instances which meet the rule and their mean value

iv) **complexity**: the length of a rule, measured as the number of variable-value pairs of a condition

Rules are then **pruned** based on a leave-one-out heuristic on each atomic condition, to reduce the effect of pruning on the rule quality

Finally, rules are **selected** by applying feature selection (*building a rules x instances boolean matrix, where each entry corresponds to instance j satisfying rule i*) to the conditions from a tree ensemble. In this way, a compact rule set that contains relevant and non-redundant rules is obtained.

BELLATREX

2023

IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 21 March 2023, accepted 29 March 2023, date of publication 20 April 2023, date of current version 2 May 2023.

Digital Object Identifier 10.1109/ACCESS.2023.3268866

RESEARCH ARTICLE

BELLATREX: Building Explanations Through a LocaLly AccuraTe Rule EXtractor

KLEST DEDJA[1,2], FELIPE KENJI NAKANO[1,2], KONSTANTINOS PLIAKOS[3], AND CELINE VENS[1,2]

[1]Department of Public Health and Primary Care, KU Leuven, 8500 Kortrijk, Belgium
[2]ITEC, imec Research Group at KU Leuven, 8500 Kortrijk, Belgium
[3]Department of Management, Strategy, and Innovation, KU Leuven, 3000 Leuven, Belgium

Corresponding author: Celine Vens (celine.vens@kuleuven.be)

This work was supported in part by the Flemish Government (AI Research Program), and in part by the Research Fund Flanders under Project G080118N and Project G0A2120N.

ABSTRACT Random forests are machine learning methods characterised by high performance and robustness to overfitting. However, since multiple learners are combined, they are not as interpretable as a single decision tree. In this work we propose a novel method that is Building Explanations through a LocalLy AccuraTe Rule EXtractor (Bellatrex), which is able to explain the forest prediction for a given test instance with only a few diverse rules. Starting from the decision trees generated by a random forest, our method: 1) pre-selects a subset of the rules used to make the prediction; 2) creates a vector representation of such rules; 3) projects them to a low-dimensional space; 4) clusters such representations to pick a rule from each cluster to explain the instance prediction. We test the effectiveness of Bellatrex on 89 real-world datasets and we demonstrate the validity of our method for binary classification, regression, multi-label classification and time-to-event tasks. To the best of our knowledge, it is the first time that an interpretability toolbox can handle all these tasks within the same framework. We also show that Bellatrex is able to approximate the performance of the corresponding ensemble model in all considered tasks, and it does so while selecting at most three rules from the whole forest. Finally, a comparison with similar methods in literature also shows that our proposed approach substantially outperforms other explainable toolboxes in terms of predictive performance.

MAIN IDEA

Derive a set of few diverse rules to explain the RF prediction for a given test instance

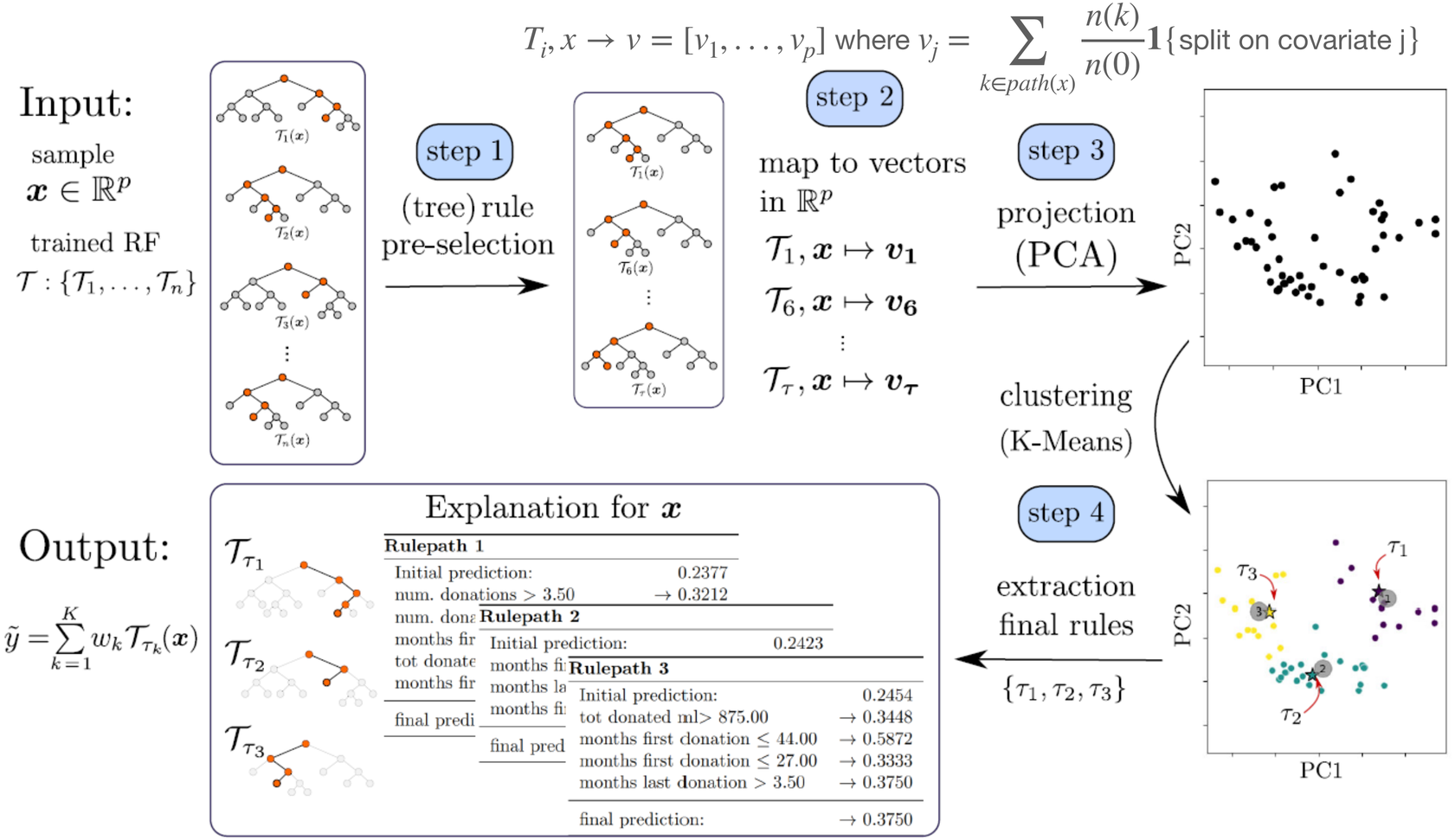
BELLATREX

BELLATREX can be applied to multiple problems such as binary **classification**, **regression**, **multi-label classification** and **time-to-event tasks** and is a **local approach** for explaining RF predictions. The methodology is built on top of a trained **random forest** and consists of four main steps:

1) for a given input instance, pre-select a subset of the rules used to make the prediction based on how close their prediction is with respect to the original ensemble
2) create a vector representation of such rules by extracting information from the rule structure
3) project them to a low-dimensional space
4) cluster such representations to pick a rule from each cluster to explain the instance prediction

These final rules are then presented to the end user as an explanation for the forest prediction

BELLATREX


$T_i, x \to v = [v_1, \ldots, v_p]$ where $v_j = \sum_{k \in path(x)} \frac{n(k)}{n(0)} \mathbf{1}\{\text{split on covariate j}\}$
Input:
sample
$\boldsymbol{x} \in \mathbb{R}^p$
trained RF
$\mathcal{T} : \{\mathcal{T}_1, \ldots, \mathcal{T}_n\}$
step 1
(tree) rule pre-selection
step 2
map to vectors in $\mathbb{R}^p$
$\mathcal{T}_1, \boldsymbol{x} \mapsto \boldsymbol{v_1}$
$\mathcal{T}_6, \boldsymbol{x} \mapsto \boldsymbol{v_6}$
$\mathcal{T}_\tau, \boldsymbol{x} \mapsto \boldsymbol{v_\tau}$
step 3
projection (PCA)
PC2
PC1
clustering (K-Means)
step 4
extraction final rules
$\{\tau_1, \tau_2, \tau_3\}$
Output:
$\tilde{y} = \sum_{k=1}^{K} w_k \mathcal{T}_{\tau_k}(\boldsymbol{x})$
Explanation for $\boldsymbol{x}$
Rulepath 1
Initial prediction: 0.2377
num. donations > 3.50 → 0.3212
Rulepath 2
Initial prediction: 0.2423
Rulepath 3
Initial prediction: 0.2454
tot donated ml> 875.00 → 0.3448
months first donation ≤ 44.00 → 0.5872
months first donation ≤ 27.00 → 0.3333
months last donation > 3.50 → 0.3750
final prediction: → 0.3750

THIS IS THE END



Thanks for the attention!

References

[1] Quinlan, J. R. ”C4.5: Programs for Machine Learning”. Morgan Kaufmann Publishers, (1993)

[2] Mark W. Craven and Jude W. Shavlik, ‘Extracting tree-structured representations of trained networks’, in NIPS 1995, pp. 24–30. MIT Press, (1995)

[3] Domingos, Pedro. "Knowledge discovery via multiple models." Intelligent Data Analysis 2.1-4 (1998): 187-202.

[4] Thiagarajan, Jayaraman J., et al. "Treeview: Peeking into deep neural networks via feature-space partitioning." arXiv preprint arXiv:1611.07429 (2016).

[5] Zhou, Yichen, and Giles Hooker. "Interpreting models via single tree approximation." arXiv preprint arXiv:1610.09036 (2016).

[6] Guidotti, Riccardo, et al. "Local rule-based explanations of black box decision systems." arXiv preprint arXiv:1805.10820 (2018).

[7] Deng H “Interpreting tree ensembles with intrees”. Int J Data Sci Anal 7(4) (2019): 277-287.

[8] Dedja, Klest, et al. "BELLATREX: Building explanations through a locally accurate rule extractor." Ieee Access 11 (2023): 41348-41367.